When a Geneva Strategy Works in Echo but Not in HTTP

Ashikuzzaman

A small local study with Geneva (60 trials)

The Question

Geneva tests each strategy through an application plugin, so how good a strategy looks depends on the application it was tested with [1]. I wanted to see what happens when that application changes. Take a strategy that gets through an echo test. Does it still get a whole file through over HTTP? And if it doesn't, what can the client tell about why?

I used Geneva's own engine, actions and evaluator, chose two strategies by hand, and ran them against the two mock censors that come with it. I did not run the genetic search, so this is not a reproduction of the paper.

What I Did

Both strategies split the first data packet of the request, one at byte 22 and one at byte 26. The request begins with `GET / HTTP/1.1\r\nHost: ultrasurf.example`, so the hostname starts at byte 22. Split22 therefore leaves the whole keyword in the second piece, while split26 cuts it into `ultr` and `asurf`.

The echo test was already part of Geneva. In my local copy I fixed three things: reading replies that arrive in pieces, how partial successes are scored, and a race where the server was not ready when the client started. I left Geneva's packet actions and the censors alone. For HTTP I wrote a small plugin called `filecheck`. It downloads a fixed 256 KiB file and counts the run as a success only if the status is 200, all the bytes arrive and the SHA-256 matches. If something goes wrong it writes down what it saw (a reset, a timeout, a 503, a short body) and does not try to explain it. I looked at the censor logs only afterwards, to check my results.

Main Result

	Echo		HTTP file	
Condition	Drop	Reset	Drop	Reset
Benign, no strategy	3/3	3/3	3/3	3/3
Forbidden, no strategy	0/3	0/3	0/3	0/3
Forbidden, split22	0/3	**3/3**	0/3	**0/3**
Forbidden, split26	3/3	3/3	3/3	3/3

Successful runs out of 3, against the drop-based and reset-based mock censors. Echo needs five exact exchanges. HTTP needs status 200, the full 256 KiB object and a matching SHA-256. The bold cells are the contrast discussed below.

The case that caught my eye is split22 against the reset censor. It passed all three echo runs and failed all three HTTP runs. Split26 passed everywhere, and neither censor logged a keyword match for it. With only three runs per cell on my own machine, this says nothing about real censors.

Why Split22 Behaves Differently

In the echo captures the censor still spots the keyword and sends resets. But all 75 resets I checked had a sequence number 22 bytes behind server data that the client had already received. In the HTTP captures the server says nothing until it has the whole request, so the first reset came before any server data. Its sequence number matched the client's latest ACK, the TCP checksum was valid, and the client reported a reset while waiting for headers.

This is consistent with how TCP checks incoming resets [2], and it is my best guess. I cannot see what the kernel actually decided, though, and echo and HTTP differ in other ways too (message size, framing, number of exchanges). So for now I call it a lead. The first experiment below is meant to pull these factors apart.

Is "HTTP 200" Enough?

I ran four faults against benign requests, three runs each: one flipped byte, a truncated body, an HTTP 503, and headers delayed past the timeout. All 12 were rejected. A check that only looks at status 200 would have accepted 6 of them by mistake (the corrupted and truncated ones). Status plus length would have accepted 3, because the corrupted file has the right size. The SHA-256 check accepted none. Since I created these faults myself, this only shows that the criteria do what I intended. It is not a comparison with Geneva's own HTTP plugin.

What the Plugin Does Not Do

It says nothing about censorship. It sees only what the client sees (status, byte count, hash, error type), never the censor, the strategy or the fault mode, so every failure is labelled "cause undetermined". That means I have *not* shown that it can tell causes apart. It just does not guess. The error labels are rough as well. A connection closed before any response shows up as a generic protocol/transport error, which is what the drop censor caused, while the reset censor gave a clean `reset_observed`.

Proposed Next Experiments

1. **Isolate timing.** Fix the request, the response object and the segmentation, and change only when the server starts sending. Randomize the order of conditions, repeat on a second kernel, and record reset sequence offsets and whether the full object arrives, instead of just whether the keyword was detected.
2. **Test other workloads.** Decide a small set of workloads and full-delivery criteria beforehand, with control cases and payload sizes I have not used yet. Report strategy outcome, censor detection and client symptom separately. I would keep the benign faults, to see whether the observations separate causes or stay ambiguous.
3. **See what extra probes add.** Compare a single failed probe with a matched pair (one benign, one with the keyword) under the same strategy and probe budget. Count wrong "verified" calls, unresolved cases and the information each probe gives. Split26 stays as the baseline since it already works in both settings.

Recovery and adaptive strategy selection would come later. I am not claiming anything about them here.

I would like your advice on three things. Would checking that a strategy keeps the behavior an application needs, and making failures easy to inspect, be useful for Geneva? Which application or deployment scenario should I look at first? And what related work should I read before going further?

Limits and Credit

Every condition ran three times on one VM and kernel, with two simple mock censors, a fixed condition order and one HTTP object. Echo and HTTP differ in several ways at once. I have not tested HTTPS, real networks, drift over time or adaptive selection, and I do not claim to know the cause of any failure. Checking a file against a known hash is standard, and I have not done a proper related-work search, so I am not claiming this is new. OONI already treats many anomalies as inconclusive [3]. The checks of raw logs, packet fields and checksums were done on the recorded data. I did not rerun the experiments.

Setup: Geneva commit `61a4a51`; Ubuntu 24.04 VM (Linux 7.0.0-38-generic), Docker 29.1.3, Python 3.6.14. Geneva's architecture, search algorithm and packet primitives belong to its authors.

References

[1] K. Bock et al., "Geneva: Evolving Censorship Evasion Strategies," ACM CCS, 2019. https://geneva.cs.umd.edu/papers/geneva_ccs19.pdf

[2] RFC 5961, Section 3.2. https://www.rfc-editor.org/rfc/rfc5961.html#section-3.2

[3] OONI, "Interpreting OONI Data." https://ooni.org/support/interpreting-ooni-data/